Contents

1 Introduction **3**

2 Related Work **4**
2.1 Sparse Mixture of Experts . . . 4
2.2 Shared Experts and Structured Routing . . . 4
2.3 Parameter-Efficient Transfer Learning . . . 4
2.4 Router Stability and Noise Injection . . . 5
2.5 Fused Kernels and Systems Optimizations . . . 5

3 Expert Topology **5**
3.1 The Three Axes of Structure . . . 5
3.2 Generation: Hierarchical Budget . . . 6
3.3 Flavor: Functional Specialization . . . 6
3.4 Channel: Multi-View Representation . . . 6
3.5 Topological Invariants . . . 7

4 Parameter Factorization: Core + LoRA **7**
4.1 Factorization Structure . . . 7
4.2 Initialization: Kaiming with Scaling . . . 7
4.3 Kato Perturbation Theory Interpretation . . . 8
4.4 Transfer Learning Protocol . . . 8

5 Multi-Channel Routing **8**
5.1 Generation Routing: Consensus . . . 8
5.2 Flavor Routing: Diversification . . . 9
5.3 Hybrid Routing: Expert Choice + Token Choice . . . 9

6 Budget Control via Lagrangian Duality **10**
6.1 Problem Formulation . . . 10
6.2 Lagrangian Dual Variable with Two-Phase Update . . . 10

7 Covariance Gradient Mixing **11**
7.1 The Expert Isolation Problem . . . 11
7.2 Gradient Mixing Mechanism . . . 11
7.3 Convergence Guarantee . . . 11
7.4 Memory-Efficient Implementation . . . 12

8 Router Stability: GARN **12**
8.1 Router Collapse as Dynamical System . . . 12
8.2 Gradient-Adaptive Router Noise . . . 12
8.3 Stability Analysis . . . 13

9 Path Consensus and Cross-Layer Coherence **13**
9.1 Motivation . . . 13
9.2 Path Consensus Loss . . . 13
9.3 Soft Constraint via Lagrangian . . . 14
9.4 Relationship to Curriculum Logic . . . 14

10 Mathematical Foundations **14**
10.1 Master Functional Equation . . . 14
10.2 Convergence of the Full Objective . . . 15
10.3 Operator-Theoretic View: Core + LoRA as Perturbation . . . 15

11 Ontogenic Pre-training and Category-Theoretic Bridge **16**
11.1 From "Zero Quarks" to Language . . . 16
11.2 Phase 0: Synthetic Quarks . . . 16
11.3 Phase 1: Language Immersion . . . 16
11.4 Category-Theoretic Formalization . . . 17

12 Systems Design: CUDA and Data-Oriented Design **17**
12.1 Data-Oriented Design Principles . . . 17
12.2 Fused Grouped GEMM . . . 18
12.3 Persistent Kernels: FlashDMoE Integration . . . 18
12.4 Fused Computation-Communication via NVSHMEM . . . 19

13 Training Methodology **19**
13.1 Three-Phase Training Protocol . . . 19
13.1.1 Phase 0: Ontogenic Pre-training (Quark-Level) . . . 19
13.1.2 Phase 1: Language Immersion . . . 19
13.1.3 Phase 2: Domain Transfer (Optional) . . . 20
13.2 Anti-Forgetting: Activation-Importance Consolidation (AIC) . . . 20
13.3 Routing Replay Buffer . . . 21

14 Reference Implementation **21**
14.1 GradientMixing (Dimension-Preserving, Per-Channel) . . . 21
14.2 ASMoELayer (Vectorized Forward) . . . 22
14.3 Routing Replay Buffer . . . 25

15 Experimental Design **26**
15.1 Proposed Benchmarks . . . 26
15.2 Ablation Studies . . . 26

16 Discussion **27**
16.1 The Quark Analogy: From Metaphor to Method . . . 27
16.2 Known Implementation Issues . . . 27
16.3 Limitations and Open Questions . . . 28

17 Conclusion **29**

A Engineering PRD: AS-MoE Production System **29**
A.1 Overview . . . 29
A.2 System Requirements . . . 29
A.3 Architecture Components . . . 29
A.3.1 Training Pipeline . . . 29
A.3.2 Inference Pipeline . . . 29
A.4 Configuration Schema . . . 29
A.5 Milestones . . . 31

B Notation Reference **31**

1 Introduction

Mixture of Experts (MoE) architectures have emerged as the dominant paradigm for scaling transformer-based language models beyond the computational constraints of dense networks. By activating only a sparse subset of parameters per token, MoE models achieve parameter counts in the trillions while maintaining inference costs comparable to dense models orders of magnitude smaller. However, the fundamental design of MoE layers—particularly the questions of *how experts should be organized*, *how tokens should be routed*, and *how expert specialization should be maintained during training*—remains largely ad hoc. Current architectures typically treat experts as an unstructured bag of interchangeable feed-forward networks, relying on learned routers whose failure modes (load imbalance, representation collapse, and training instability) are well-documented but poorly addressed from first principles.

This paper introduces **AS-MoE** (*Adaptive Structured Mixture of Experts*), an architecture that imposes a principled topological and algebraic structure on the expert ensemble. AS-MoE organizes experts along three orthogonal axes—**Generation** (g), **Flavor** (f), and **Channel** (k)—derived from a formal analogy with the quark model of particle physics, but justified independently through considerations of computational budgeting, functional specialization, and multi-view representation learning. The key insight is that the "color confinement" analogy from physics—quarks cannot be isolated, only observed in bound states—translates to a concrete architectural inductive bias: experts should not be independent, free-floating parameters, but rather *correlated components of a structured decomposition*, constrained by shared cores and mixed gradients.

The contributions of this paper are:

1. A **topologically structured expert organization** (Section 3) that decouples computational budget (Generation), functional specialization (Flavor), and representational diversity (Channel) into orthogonal axes, with the g=0 generation serving as a universal shared expert analogous to DeepSeek's Shared Expert mechanism.

2. A **Core + LoRA parameter factorization** (Section 4) that separates universal, transferable core weights from view-specific, low-rank adaptation matrices, enabling efficient transfer learning without router invariance constraints.

3. A **Lagrangian budget control** mechanism (Section 6) that enforces strict FLOP constraints via dual variable optimization with exponential moving average updates, providing hard computational guarantees absent in softmax-gating approaches.

4. A **covariance gradient mixing** scheme (Section 7) that prevents expert isolation and representation collapse, with convergence guarantees formalized via Gershgorin circle analysis.

5. A **Gradient-Adaptive Router Noise** (GARN) mechanism (Section 8) that stabilizes router dynamics as a stochastic perturbation of the routing ODE, preventing mode collapse without sacrificing specialization.

6. A **Path Consensus Loss** (Section 9) that enforces cross-layer routing consistency, inspired by PathMoE, ensuring stable expert trajectories through network depth.

7. A **category-theoretic formalization** of ontogenic pre-training (Section 11) that defines the compositional bridge from attribute-level "quark" representations to language-level semantics as a structure-preserving functor.

8. A **systems design** (Section 12) grounded in Data-Oriented Design principles, including Structure-of-Arrays memory layout, fused Grouped GEMM kernels, persistent kernels via FlashDMoE, and fused computation-communication via NVSHMEM.

9. A comprehensive **training methodology** (Section 13) incorporating ontogenic pre-training, EWC++ for catastrophic forgetting prevention, and routing replay buffers.

The remainder of this paper is organized as follows. Section 2 surveys related work. Sections 3–9 detail the core architecture. Section 10 develops the mathematical foundations. Section 11 presents the ontogenic pre-training framework. Section 12 covers systems design. Section 13 describes the full training methodology. Section 14 provides reference implementations. Appendix A contains the engineering PRD.

2 Related Work

2.1 Sparse Mixture of Experts

The MoE paradigm traces its origins to Jacobs et al. (1991) and Jordan and Jacobs (1994), who introduced competitive learning among expert networks gated by a learned router. The modern instantiation in transformer architectures was pioneered by Shazeer et al. (2017) with the Sparsely-Gated MoE layer, which demonstrated that top-k routing could scale model capacity with minimal per-token cost. Subsequent work—Switch Transformers (Fedus et al., 2022), GShard (Lepikhin et al., 2021), and the Suite of MoE models—established the standard pattern: a linear projection router producing softmax weights, followed by top-k selection and weighted expert computation.

A persistent challenge in MoE architectures is *load imbalance*: routers tend to collapse onto a small subset of experts, leaving others untrained. Auxiliary losses such as the load-balancing loss (Fedus et al., 2022) and the z-loss (Zoph et al., 2022) provide soft penalties but offer no hard guarantees on FLOP budgets. Expert Choice routing (Zhou et al., 2022) inverts the routing direction—experts select their top tokens rather than tokens selecting experts—achieving perfect load balance by construction. However, Expert Choice is fundamentally incompatible with autoregressive generation, where tokens arrive sequentially and cannot be preemptively ranked. AS-MoE addresses this through a *hybrid routing* strategy (Section 5): Expert Choice during pre-training for load-balanced optimization, and Token Choice during inference for causal compatibility.

2.2 Shared Experts and Structured Routing

DeepSeek-V2 (Dai et al., 2024) introduced the Shared Expert mechanism, reserving a subset of experts that process *every* token regardless of routing decisions. This architectural innovation provides a universal representational backbone that captures common knowledge, while routed experts handle specialization. AS-MoE generalizes this concept through its Generation axis: $g{=}0$ serves as the Shared Expert, but higher generations ($g{=}1, 2, \ldots$) provide progressively larger computational budgets for tokens requiring deeper processing. This hierarchical budget structure is absent in DeepSeek, where the shared/routed distinction is binary.

PathMoE (2024) introduced the concept of *path consistency* across layers, encouraging tokens to follow stable expert trajectories through network depth rather than independently routing at each layer. This reduces representational fragmentation and improves training stability. AS-MoE incorporates this insight through the Path Consensus Loss (Section 9), which aligns routing distributions across adjacent layers while preserving the flexibility to re-route when representational demands change.

2.3 Parameter-Efficient Transfer Learning

LoRA (Hu et al., 2022) demonstrated that low-rank adaptation matrices applied to frozen pre-trained weights can achieve transfer performance comparable to full fine-tuning at a fraction of the parameter cost. The IniLoRA analysis (2024) further established that proper initialization

of LoRA matrices—specifically, Kaiming initialization with appropriate scaling—is critical for stable training. AS-MoE integrates LoRA not as a post-hoc adaptation mechanism, but as a *first-class architectural component*: every expert is factored into a universal Core (shared across views and frozen during transfer) and a view-specific LoRA adapter (trainable). This design eliminates the need for router invariance during transfer, as the Core provides a stable representational anchor.

2.4 Router Stability and Noise Injection

Router collapse—where a single expert receives all tokens—is a well-known failure mode. Prior solutions include load-balancing auxiliary losses (Fedus et al., 2022), jitter noise on router logits (Lepikhin et al., 2021), and expert capacity constraints (Lewis et al., 2021). However, these are heuristic patches without formal stability guarantees. The GARN mechanism introduced in this paper (Section 8) provides a principled approach: router noise is derived from the gradient of the routing distribution, ensuring that perturbation magnitude scales with the risk of collapse. This is formalized through the lens of stochastic stabilization of dynamical systems (Arnold, 2013).

2.5 Fused Kernels and Systems Optimizations

Recent systems work has significantly improved MoE throughput. Megablocks (Gale et al., 2023) demonstrated that sparse MoE can be reformulated as block-sparse operations amenable to GPU optimization. SonicMoE and FlashDMoE (2024) introduced persistent kernels that reduce kernel launch overhead and improve memory locality for expert computation. Grouped GEMM (2024) enables efficient batched matrix multiplication across expert groups with varying dimensions. AS-MoE's systems design (Section 12) builds on these advances with a Structure-of-Arrays memory layout optimized for the Generation/Flavor/Channel topology, and fused computation-communication via NVSHMEM for distributed training.

3 Expert Topology

3.1 The Three Axes of Structure

The AS-MoE expert ensemble is indexed by three orthogonal coordinates:

Definition 3.1 (Expert Index)**.** An expert $E_{g,f,k}$ is identified by the triple (g, f, k) where:

- $g \in \{0, 1, \ldots, G-1\}$ is the **Generation**, encoding the computational budget level. Generation g=0 is the *Shared Expert*, active for all tokens. Higher generations allocate progressively more FLOPs.
- $f \in \{1, 2, \ldots, F\}$ is the **Flavor**, encoding functional specialization (e.g., syntax, semantics, world knowledge).
- $k \in \{1, 2, \ldots, K\}$ is the **Channel** (or View), encoding representational diversity via independent projection heads.

The total number of experts is $N_e = G \times F \times K$. A typical configuration is G=3, F=4, K=2, yielding N_e=24 experts per layer, of which only a sparse subset is activated per token.

3.2 Generation: Hierarchical Budget

The Generation axis implements a hierarchical computational budget. Each generation g is associated with an intermediate dimension $d_{\text{ff}}^{(g)}$ and a FLOP budget $B^{(g)}$:

$$d_{\text{ff}}^{(g)} = d_{\text{base}} \cdot 2^g, \quad g = 0, 1, \ldots, G{-}1, \tag{1}$$

$$B^{(g)} \propto d_{\text{ff}}^{(g)} \cdot d_{\text{model}}, \tag{2}$$

where d_{base} is the minimum FFN dimension (e.g., 1024) and d_{model} is the model hidden dimension. For $G{=}3$:

$$d_{\text{ff}}^{(0)} = 1024, \quad d_{\text{ff}}^{(1)} = 2048, \quad d_{\text{ff}}^{(2)} = 4096. \tag{3}$$

The *Shared Expert* ($g{=}0$) processes every token with minimal computation, providing universal representational coverage. Tokens requiring deeper processing are routed to higher generations by the generation router. This design ensures that simple tokens (common patterns, high-frequency words) consume minimal FLOPs, while complex tokens (rare words, mathematical expressions, code) receive proportionally more compute.

Remark 3.1. The Generation hierarchy is analogous to the energy level structure in quantum mechanics: low-energy states (simple tokens) occupy ground-level generations, while high-energy excitations (complex tokens) are promoted to higher generations. The Lagrangian budget controller (Section 6) acts as the "thermostat" regulating the distribution of tokens across energy levels.

3.3 Flavor: Functional Specialization

The Flavor axis encodes functional specialization. Unlike Generation, which is determined by computational demand, Flavor is determined by the *type* of processing a token requires. The flavor router produces a distribution over F flavors, and the top-m flavors are selected per token (typically $m{=}1$ or $m{=}2$).

The critical design choice is that Flavor routing is *independent per channel*: each of the K channels computes its own flavor distribution. This allows the same token to receive different functional treatments in different representational subspaces, which is essential for handling tokens with polysemous or multifaceted semantics.

3.4 Channel: Multi-View Representation

The Channel axis implements multi-view representation learning. Each channel k applies an independent linear projection $\mathbf{W}_k \in \mathbb{R}^{d_{\text{model}} \times d_{\text{proj}}}$ to the input, producing a view-specific representation:

$$\mathbf{h}_k = \mathbf{x}\mathbf{W}_k, \quad k = 1, \ldots, K. \tag{4}$$

Routing and expert computation proceed independently within each channel, and the final output is the average across channels:

$$\text{Output} = \frac{1}{K} \sum_{k=1}^{K} \text{Channel}_k(\mathbf{h}_k). \tag{5}$$

An orthogonal projection loss $\mathcal{L}_{\text{ortho}}$ encourages channels to capture distinct representational subspaces:

$$\mathcal{L}_{\text{ortho}} = \sum_{k \neq k'} \left\| \mathbf{W}_k^\top \mathbf{W}_{k'} \right\|_F^2. \tag{6}$$

3.5 Topological Invariants

The three-axis structure gives rise to topological invariants that constrain the expert ensemble:

Proposition 3.1 (Budget Monotonicity)**.** For any two generations $g < g'$, the per-expert FLOP budget satisfies $B^{(g)} \leq B^{(g')}$, and the set of tokens routed to g' is a subset of those routed to g under the Lagrangian controller.

This follows from the curriculum logic (Section 5): a token must "pass through" lower generations before being promoted, ensuring that the Shared Expert always contributes to the representation.

Proposition 3.2 (Flavor Coverage)**.** With F flavors and top-m selection, every token is processed by at least m functional specialists, and the expected number of active flavors per batch is $\min(F, m \cdot B/|\text{batch}|)$ under load balance.

4 Parameter Factorization: Core + LoRA

4.1 Factorization Structure

Each expert $E_{g,f,k}$ factorizes its weight matrices into a **Core** (universal, shared across views) and a **LoRA adapter** (view-specific):

Definition 4.1 (Core + LoRA Factorization)**.** For expert $E_{g,f,k}$ with FFN weight matrices $\mathbf{W}_{\text{up}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}^{(g)}}$ and $\mathbf{W}_{\text{down}} \in \mathbb{R}^{d_{\text{ff}}^{(g)} \times d_{\text{model}}}$:

$$\mathbf{W}_{\text{up}} = \underbrace{\mathbf{W}_{\text{up}}^{\text{core}}}_{\text{shared, frozen}} + \underbrace{\mathbf{A}_{\text{up}}^{(g,f,k)} \mathbf{B}_{\text{up}}^{(g,f,k)}}_{\text{LoRA, trainable}}, \tag{7}$$

$$\mathbf{W}_{\text{down}} = \underbrace{\mathbf{W}_{\text{down}}^{\text{core}}}_{\text{shared, frozen}} + \underbrace{\mathbf{A}_{\text{down}}^{(g,f,k)} \mathbf{B}_{\text{down}}^{(g,f,k)}}_{\text{LoRA, trainable}}, \tag{8}$$

where $\mathbf{A} \in \mathbb{R}^{d \times r}$, $\mathbf{B} \in \mathbb{R}^{r \times d'}$, and $r \ll d$ is the LoRA rank (typically r=4 or r=8).

The Core weights are shared across all (f, k) within the same generation g and frozen during transfer learning. The LoRA adapters are specific to each (g, f, k) triple and carry the view- and specialization-specific adaptations.

4.2 Initialization: Kaiming with Scaling

Following the IniLoRA analysis, proper initialization of LoRA matrices is critical for training stability. We adopt:

1. $\mathbf{A}$ is initialized with Kaiming uniform: $\mathbf{A}_{ij} \sim \mathcal{U}\left(-\sqrt{3/d}, \sqrt{3/d}\right)$.
2. $\mathbf{B}$ is initialized to zero: $\mathbf{B} = \mathbf{0}$.
3. A scaling factor α/r is applied to the LoRA output, where α is a hyperparameter (typically α=16).

The zero initialization of $\mathbf{B}$ ensures that the expert output is purely determined by the Core at initialization, with LoRA contributions growing smoothly from zero. The Kaiming initialization of $\mathbf{A}$ preserves the variance of activations through the low-rank projection, preventing signal attenuation or explosion in the early stages of training.

4.3 Kato Perturbation Theory Interpretation

The Core + LoRA factorization admits a natural interpretation through Kato's perturbation theory for linear operators. Consider the Core weight $\mathbf{W}^{\text{core}}$ as the *unperturbed operator* and the LoRA term $\mathbf{AB}$ as the *perturbation*:

$$\mathbf{W} = \mathbf{W}^{\text{core}} + \epsilon \cdot \Delta\mathbf{W}, \quad \Delta\mathbf{W} = \mathbf{AB}, \quad \epsilon = \frac{\alpha}{r}. \tag{9}$$

Kato's theory provides bounds on how the spectral properties of $\mathbf{W}$ deviate from $\mathbf{W}^{\text{core}}$ under perturbation. Specifically, if λ_i are the eigenvalues of $\mathbf{W}^{\text{core}}$, then the perturbed eigenvalues λ_i' satisfy:

$$|\lambda_i' - \lambda_i| \leq \|\Delta\mathbf{W}\|_2 \cdot \frac{\alpha}{r} + O\left(\left(\frac{\alpha}{r}\right)^2\right). \tag{10}$$

This guarantees that the LoRA adaptation does not drastically alter the spectral structure of the Core, which is critical for maintaining stable representations during transfer. The low-rank structure of the perturbation ($\text{rank}(\Delta\mathbf{W}) = r \ll d$) further constrains the perturbation to a low-dimensional subspace, limiting its effect on the operator's dominant eigenvectors.

4.4 Transfer Learning Protocol

During transfer to a new domain or task:

1. The Core weights $\mathbf{W}^{\text{core}}$ are **frozen**.
2. All LoRA adapters are **re-initialized** (or selectively reset).
3. The generation and flavor routers are **re-initialized**, allowing the new domain to develop its own routing topology.
4. Channel projections $\mathbf{W}_k$ may be **fine-tuned** with a reduced learning rate.

This protocol eliminates the need for *router invariance*—the constraint that the same tokens must be routed to the same experts across domains—because the Core provides a domain-agnostic representational foundation. The LoRA adapters learn domain-specific adaptations *within* the subspace defined by the Core, and the routers are free to discover new specialization patterns.

5 Multi-Channel Routing

5.1 Generation Routing: Consensus

The generation router determines the computational budget level for each token. Unlike flavor routing, which can differ across channels, generation routing enforces *cross-channel consensus*: all channels agree on the generation level for a given token.

Definition 5.1 (Generation Routing)**.** For input $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$, the generation logits are computed as:

$$\mathbf{z}_g = \mathbf{x}\mathbf{W}_g + \boldsymbol{\phi}_g, \quad \mathbf{W}_g \in \mathbb{R}^{d_{\text{model}} \times G}, \quad \boldsymbol{\phi}_g \in \mathbb{R}^G, \tag{11}$$

where $\boldsymbol{\phi}_g$ is a learnable bias. The generation distribution is:

$$\pi_g = \text{softmax}(\mathbf{z}_g / \tau_g), \tag{12}$$

with temperature $\tau_g > 0$. The generation assignment is:

$$g^* = \text{argmax}_g \, \pi_g. \tag{13}$$

During training, a Straight-Through Estimator (STE) is used: the forward pass uses the hard assignment g^*, while the backward pass flows through the soft distribution π_g. This ensures differentiability while maintaining sparse activation.

Curriculum Logic. A key inductive bias is that tokens must "graduate" through generations. A token at generation g receives contributions from *all* generations $\leq g$:

$$\text{Output}_{\text{gen}} = \sum_{g'=0}^{g^*} \text{Expert}_{g'}(\mathbf{x}). \tag{14}$$

This curriculum ensures that the Shared Expert (g=0) always contributes, providing a stable representational baseline, while higher generations add incremental specialization.

5.2 Flavor Routing: Diversification

Within each channel k and generation g, the flavor router determines which functional specialists process the token:

Definition 5.2 (Flavor Routing)**.** For the projected representation $\mathbf{h}_k = \mathbf{x}\mathbf{W}_k$ within channel k and generation g:

$$\mathbf{z}_f^{(g,k)} = \mathbf{h}_k \mathbf{W}_f^{(g,k)} + \boldsymbol{\phi}_f^{(g,k)}, \tag{15}$$

$$\pi_f^{(g,k)} = \text{softmax}(\mathbf{z}_f^{(g,k)}/\tau_f), \tag{16}$$

$$\mathcal{S}_f^{(g,k)} = \text{top-}m\left(\pi_f^{(g,k)}\right), \tag{17}$$

where $\mathcal{S}_f^{(g,k)}$ is the set of m selected flavors. The flavor weights are renormalized over the selected set. Critically, the flavor router is conditioned on both channel k and generation g: different generations may assign different functional specializations to the same token, reflecting the fact that computational depth interacts with functional type.

A **diversity loss** encourages balanced flavor utilization within each generation-channel pair:

$$\mathcal{L}_{\text{div}} = \frac{1}{G \cdot K} \sum_{g=1}^{G} \sum_{k=1}^{K} \left[\frac{1}{F} \sum_{f=1}^{F} \left(\bar{\pi}_f^{(g,k)} - \frac{1}{F} \right)^2 \right], \tag{18}$$

where $\bar{\pi}_f^{(g,k)}$ is the average routing probability for flavor f across the batch, computed independently for each generation g and channel k. Averaging per (g, k) prevents mixing different routing distributions across generations, which would obscure imbalances specific to a particular budget level.

5.3 Hybrid Routing: Expert Choice + Token Choice

A critical practical consideration is the incompatibility of Expert Choice (EC) routing with autoregressive generation. EC routing allows experts to select their top tokens, achieving perfect load balance by construction. However, EC requires access to the full token sequence before routing, which is impossible during causal inference where tokens arrive one at a time.

AS-MoE resolves this with a **Hybrid Routing** strategy:

1. **Pre-training phase**: Expert Choice routing is used for both generation and flavor assignment. This maximizes training efficiency and load balance.

2. **Fine-tuning / Inference phase**: Token Choice routing (standard top-k) is used, with the routers initialized from the EC-trained weights. The transition from EC to TC is smoothed by a brief fine-tuning period with TC routing and an auxiliary load-balancing loss.

This hybrid approach leverages the training efficiency of EC while ensuring causal compatibility at inference time.

6 Budget Control via Lagrangian Duality

6.1 Problem Formulation

The goal is to enforce a strict FLOP budget $\mathcal{B}$ per token, where:

$$\mathcal{B} = \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}\left[\sum_{g,f,k} \mathbb{1}[E_{g,f,k} \text{ active}] \cdot B^{(g)}\right]. \tag{19}$$

Standard load-balancing losses provide soft penalties but no hard guarantees. We formulate budget control as a constrained optimization problem:

$$\min_{\boldsymbol{\theta}} \mathcal{L}_{\text{task}}(\boldsymbol{\theta}) \quad \text{s.t.} \quad \mathcal{B}(\boldsymbol{\theta}) \leq B_{\text{target}}. \tag{20}$$

6.2 Lagrangian Dual Variable with Two-Phase Update

Introducing a Lagrangian multiplier $\mu \geq 0$, the Lagrangian is:

$$\mathcal{L}_{\text{lagr}}(\boldsymbol{\theta}, \mu) = \mathcal{L}_{\text{task}}(\boldsymbol{\theta}) + \mu\left(\mathcal{B}(\boldsymbol{\theta}) - B_{\text{target}}\right). \tag{21}$$

The dual variable μ is updated via a two-phase procedure that separates budget estimation from multiplier optimization:

Phase A: EMA for budget estimation. The empirical budget is tracked via EMA to reduce variance:

$$\bar{\mathcal{B}}^{(t)} = \beta \cdot \bar{\mathcal{B}}^{(t-1)} + (1-\beta) \cdot \mathcal{B}_{\text{batch}}^{(t)}, \tag{22}$$

where $\beta \in [0.9, 0.999]$ is the EMA decay rate and $\mathcal{B}_{\text{batch}}^{(t)}$ is the empirical FLOP budget computed from the actual (hard) routing decisions at step t.

Phase B: Gradient step for μ. The multiplier is updated via gradient ascent on the dual objective:

$$\mu^{(t+1)} = \max\left(0,\ \mu^{(t)} + \eta_\mu\left(\bar{\mathcal{B}}^{(t)} - B_{\text{target}}\right)\right), \tag{23}$$

where $\eta_\mu > 0$ is the dual learning rate. This separation ensures that μ responds to the smoothed budget estimate (not noisy per-batch fluctuations) while performing true gradient-based optimization of the Lagrangian.

Theorem 6.1 (Budget Convergence)**.** Under the two-phase update with EMA decay $\beta \in (0,1)$ and dual learning rate η_μ satisfying $\sum_t \eta_\mu = \infty$, $\sum_t \eta_\mu^2 < \infty$ (Robbins-Monro conditions), if the empirical budget $\mathcal{B}^{(t)}$ is bounded, then $\mu^{(t)}$ converges almost surely to the optimal dual variable μ^*, and the expected budget converges:

$$\lim_{t\to\infty} \mathbb{E}\left[\bar{\mathcal{B}}^{(t)}\right] \leq B_{\text{target}} + O\left(\eta_\mu\right). \tag{24}$$

The budget loss is added to the total training objective:

$$\mathcal{L}_{\text{budget}} = \mu \cdot \left(\mathcal{B} - B_{\text{target}}\right). \tag{25}$$

When the budget is exceeded, μ increases, penalizing high-generation routing and pushing tokens toward lower (cheaper) generations. When the budget is under-utilized, μ decreases toward zero, relaxing the constraint.

7 Covariance Gradient Mixing

7.1 The Expert Isolation Problem

In standard MoE training, gradients flow only through the active experts for each token. Over time, this leads to *expert isolation*: experts converge to independent local optima, losing the ability to cooperate. In extreme cases, experts develop redundant representations (mode collapse), wasting capacity.

7.2 Gradient Mixing Mechanism

AS-MoE addresses expert isolation through **covariance gradient mixing**. The key idea is to mix the gradients of different experts proportionally to their activation correlation, ensuring that correlated experts receive similar gradient signals.

Definition 7.1 (Gradient Mixing)**.** Let $\{\mathbf{y}_1, \mathbf{y}_2, \ldots, \mathbf{y}_N\}$ be the outputs of the $N = G \times F$ active experts *within a single channel* k, with routing weights $\{w_1, w_2, \ldots, w_N\}$. The mixed output is:

$$\mathbf{y} = \sum_{i=1}^{N} w_i \mathbf{y}_i. \tag{26}$$

In the backward pass, the gradient mixing matrix $\mathbf{M}$ is computed from the activation covariance:

$$\mathbf{R}_{ij} = \frac{\langle \mathbf{y}_i, \mathbf{y}_j \rangle}{\|\mathbf{y}_i\| \cdot \|\mathbf{y}_j\|}, \quad \mathbf{M} = \mathbf{I} + \gamma \cdot \mathbf{R}, \tag{27}$$

where $\gamma > 0$ is the mixing strength. The mixed gradients are:

$$\nabla_{\mathbf{y}_i}^{\text{mixed}} = \sum_j \mathbf{M}_{ji} \cdot \nabla_{\mathbf{y}_j}^{\text{raw}}. \tag{28}$$

Remark 7.1. Gradient mixing is applied **independently within each channel**, not globally across all $G \times F \times K$ experts of the layer. This design choice is motivated by two considerations: (1) experts within the same channel operate on the same projected representation $\mathbf{h}_k = \mathbf{x}\mathbf{W}_k$ and are thus directly comparable, whereas experts across channels operate in different representational subspaces; (2) per-channel mixing incurs an $O((GF)^2)$ correlation computation per channel rather than $O((GFK)^2)$ for global mixing, a significant reduction for $K \geq 2$. The N in Definition 7.1 refers to $N = G \times F$ (experts per channel), not $N_e = G \times F \times K$ (total experts per layer). In distributed settings, a DDP all-gather may extend mixing to cross-GPU expert correlations within the same channel.

The mixing matrix $\mathbf{M}$ ensures that:

- Each expert receives its own gradient (the $\mathbf{I}$ term).
- Additionally, each expert receives a fraction γ of the gradients from correlated experts (the $\gamma\mathbf{R}$ term).
- Strongly correlated experts receive similar gradient signals, preventing drift.

7.3 Convergence Guarantee

Lemma 7.1 (Gershgorin Stability)**.** The eigenvalues λ_i of the mixing matrix $\mathbf{M} = \mathbf{I} + \gamma\mathbf{R}$ satisfy:

$$\lambda_i \in \left[1 - \gamma \sum_{j \neq i} |R_{ij}|,\ 1 + \gamma \sum_{j \neq i} |R_{ij}| \right]. \tag{29}$$

Since $|R_{ij}| \leq 1$ (normalized correlation), the eigenvalues are positive for $\gamma < 1/(N-1)$, ensuring that gradient mixing does not reverse gradient directions and preserves convergence of SGD.

This guarantee is critical: without it, aggressive mixing could destabilize training by introducing gradient directions opposite to the true gradient. The Gershgorin bound provides a safe operating range for γ.

7.4 Memory-Efficient Implementation

To prevent memory explosion from storing all expert outputs, the implementation saves activations in `bfloat16` and computes the correlation matrix $\mathbf{R}$ in reduced precision. The dimension-preserving backward pass is detailed in Section 14. In distributed settings (DDP), an all-gather operation computes the global correlation matrix across GPUs, ensuring that mixing reflects the true cross-expert correlation structure rather than a per-GPU approximation.

8 Router Stability: GARN

8.1 Router Collapse as Dynamical System

The routing process can be modeled as a discrete-time dynamical system. Let $\pi^{(t)}$ denote the routing distribution at training step t. The router update can be written as:

$$\pi^{(t+1)} = F\left(\pi^{(t)}, \nabla_{\boldsymbol{\theta}} \mathcal{L}^{(t)}\right), \tag{30}$$

where F is the update mapping determined by the optimizer and loss landscape.

Router collapse occurs when F has an attracting fixed point at a degenerate distribution $\pi^* = \mathbf{e}_i$ (all mass on a single expert i). This happens when:

1. Expert i receives more tokens $\rightarrow$ more gradient updates $\rightarrow$ better performance $\rightarrow$ even more tokens (positive feedback loop).
2. Under-trained experts cannot compete $\rightarrow$ receive no tokens $\rightarrow$ never improve (starvation).

8.2 Gradient-Adaptive Router Noise

GARN (Gradient-Adaptive Router Noise) stabilizes the routing dynamics by injecting noise whose magnitude is adapted to the gradient of the routing distribution. The key insight is that collapse is signaled by *decreasing entropy* of π, and the noise should increase precisely when entropy is low.

Definition 8.1 (GARN)**.** Given router logits $\mathbf{z} \in \mathbb{R}^N$, the GARN-perturbed logits are:

$$\tilde{\mathbf{z}}_i = z_i + \sigma_i \cdot \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, 1), \tag{31}$$

where the noise scale σ_i is:

$$\sigma_i = \sigma_{\text{base}} \cdot \exp\left(-\eta \cdot H(\pi)\right) \cdot \left(1 + \alpha \cdot |\nabla_{z_i} \mathcal{L}|\right), \tag{32}$$

$H(\pi) = -\sum_i \pi_i \log \pi_i$ is the entropy of the routing distribution, σ_{base} is the base noise level, $\eta > 0$ controls entropy sensitivity, and $\alpha > 0$ controls gradient sensitivity.

The noise scale has three components:

1. **Entropy term** $\exp(-\eta H(\pi))$: When entropy is low (imminent collapse), the noise scale *increases*, perturbing the router away from the degenerate distribution. When entropy is high (balanced routing), the noise scale *decreases*, allowing the router to specialize without interference.

2. **Gradient term** $(1 + \alpha|\nabla_{z_i}\mathcal{L}|)$: Experts with large routing gradients (those whose selection is changing rapidly) receive more noise, preventing hasty commitment to a single expert. In practice, $|\nabla_{z_i}\mathcal{L}|$ is approximated by $|z_i|$ (see Remark 8.1).

3. **Base term** σ_{base}: A minimum noise floor that ensures exploration even when entropy and gradients are small.

Remark 8.1 (Practical Approximation of the Gradient Term)**.** The gradient term $|\nabla_{z_i}\mathcal{L}|$ requires the loss gradient with respect to the router logits, which is unavailable during the forward pass. The reference implementation (Section 14) uses $|z_i|$ as a proxy: since $\nabla_{z_i}\mathcal{L} = \pi_i(\delta_{i,y} - \pi_y)$ for cross-entropy loss, and $\pi_i \propto \exp(z_i)$ for the dominant expert, logit magnitude correlates with gradient magnitude. This proxy introduces a systematic bias but preserves the qualitative behavior (experts with larger logits receive more noise). A more accurate but computationally expensive alternative is a *two-pass GARN*: first forward without noise, then backward to obtain $\nabla_z\mathcal{L}$, then a second forward with gradient-scaled noise at approximately $2\times$ the cost.

8.3 Stability Analysis

Theorem 8.1 (GARN Stabilization)**.** Consider the routing ODE:

$$\frac{d\pi}{dt} = F(\pi) + \sigma(\pi) \cdot \frac{dW}{dt}, \tag{33}$$

where W is a Wiener process and $\sigma(\pi) = \sigma_{\text{base}} \exp(-\eta H(\pi))$. If the deterministic system $\dot{\pi} = F(\pi)$ has an attracting fixed point at the degenerate distribution $\pi^* = \mathbf{e}_i$, then for sufficiently large σ_{base} and η, the stochastic system has a unique stationary distribution ρ^* with $H(\rho^*) > 0$, i.e., the stationary distribution is non-degenerate.

This result follows from the theory of stochastic stabilization (Arnold, 2013): additive noise with entropy-dependent scaling destabilizes the degenerate fixed point while preserving the stability of non-degenerate fixed points. The gradient-adaptive term provides an additional stabilizing mechanism by increasing perturbation precisely where the dynamics are most unstable.

9 Path Consensus and Cross-Layer Coherence

9.1 Motivation

In standard MoE, each layer independently routes tokens to experts. This independence can lead to *representational fragmentation*: a token that is routed to a "syntax" expert in layer 3 might be routed to a "semantics" expert in layer 5, disrupting the coherent processing pathway. PathMoE demonstrated that enforcing cross-layer routing consistency improves both training stability and final model quality.

9.2 Path Consensus Loss

AS-MoE enforces cross-layer coherence through the **Path Consensus Loss**, which penalizes deviations between routing distributions at adjacent layers:

Definition 9.1 (Path Consensus Loss)**.** Let $\pi^{(\ell)}$ and $\pi^{(\ell+1)}$ be the routing distributions (over experts) at layers ℓ and $\ell + 1$. The path consensus loss is:

$$\mathcal{L}_{\text{path}} = \frac{1}{L-1} \sum_{\ell=1}^{L-1} D_{\text{KL}}\left(\pi^{(\ell+1)} \parallel \pi^{(\ell)}\right), \tag{34}$$

where D_{KL} is the Kullback-Leibler divergence and L is the total number of MoE layers. The direction $D_{\mathrm{KL}}(\pi^{(\ell+1)} \| \pi^{(\ell)})$ is chosen because minimizing this forces the later distribution $\pi^{(\ell+1)}$ to cover the support of the earlier distribution $\pi^{(\ell)}$ (mean-seeking behavior), effectively regularizing the later layer toward the earlier one.

The KL divergence is asymmetric by design: we regularize the later layer toward the earlier one, under the assumption that early layers establish the representational pathway and later layers should refine rather than overturn it. This is consistent with the observation that early MoE layers tend to capture syntactic patterns while later layers capture semantic patterns, and disrupting this hierarchy is detrimental.

9.3 Soft Constraint via Lagrangian

The path consensus loss is incorporated as a soft constraint with a learnable weight λ_{path}, which is annealed during training:

$$\lambda_{\mathrm{path}}^{(t)} = \lambda_{\mathrm{path}}^{(0)} \cdot \min\left(1, \frac{t}{T_{\mathrm{warmup}}}\right), \tag{35}$$

where T_{warmup} is the warmup period. During early training, the path consensus constraint is weak, allowing the model to discover effective routing patterns. As training progresses, the constraint strengthens, locking in stable expert trajectories.

9.4 Relationship to Curriculum Logic

The Path Consensus Loss complements the Generation curriculum (Section 5). While the curriculum ensures that tokens pass through the Shared Expert before receiving specialized processing, the Path Consensus Loss ensures that this processing trajectory is consistent across layers. Together, they enforce two complementary stability properties:

1. **Vertical stability** (curriculum): Simple processing always precedes complex processing within a layer.
2. **Horizontal stability** (path consensus): Processing pathways are consistent across layers.

10 Mathematical Foundations

10.1 Master Functional Equation

The complete AS-MoE training objective integrates all components into a single functional:

Definition 10.1 (Master Functional)**.** The AS-MoE training objective at step t is:

$$\begin{aligned}\mathcal{L}^{(t)} = \mathcal{L}_{\mathrm{task}} + \lambda_{\mathrm{cons}}\mathcal{L}_{\mathrm{cons}} + \lambda_{\mathrm{div}}\mathcal{L}_{\mathrm{div}} + \lambda_{\mathrm{ortho}}\mathcal{L}_{\mathrm{ortho}} \\ + \mu^{(t)}\left(\mathcal{B}^{(t)} - B_{\mathrm{target}}\right) + \lambda_{\mathrm{path}}^{(t)}\mathcal{L}_{\mathrm{path}} \\ + \lambda_{\mathrm{AIC}}\mathcal{L}_{\mathrm{AIC}} + \lambda_{\mathrm{replay}}\mathcal{L}_{\mathrm{replay}},\end{aligned} \tag{36}$$

where:

- $\mathcal{L}_{\mathrm{task}}$: primary language modeling loss (cross-entropy).
- $\mathcal{L}_{\mathrm{cons}}$: generation consensus loss (encourages cross-channel agreement on generation level).
- $\mathcal{L}_{\mathrm{div}}$: flavor diversity loss (prevents flavor collapse).

- $\mathcal{L}_{\text{ortho}}$: channel orthogonality loss.
- $\mu^{(t)}(\mathcal{B}^{(t)} - B_{\text{target}})$: Lagrangian budget control.
- $\mathcal{L}_{\text{path}}$: path consensus loss.
- $\mathcal{L}_{\text{AIC}}$: activation-importance consolidation on Shared Expert (Section 13).
- $\mathcal{L}_{\text{replay}}$: routing replay buffer loss (Section 13).

10.2 Convergence of the Full Objective

Theorem 10.1 (Convergence Under Gradient Mixing + GARN)**.** Consider the AS-MoE training dynamics with gradient mixing parameter $\gamma < 1/(N-1)$ (Lemma 7.1) and GARN noise with entropy sensitivity $\eta > 0$ (Theorem 8.1). Under standard assumptions (L-smoothness of $\mathcal{L}_{\text{task}}$, bounded gradient variance, learning rate η_t satisfying Robbins-Monro conditions), the iterates $\boldsymbol{\theta}^{(t)}$ converge to a stationary point of the regularized objective:

$$\mathbb{E}\left[\|\nabla\hat{\mathcal{L}}(\boldsymbol{\theta}^{(t)})\|^2\right] \to 0 \quad \text{as } t \to \infty, \tag{37}$$

where $\hat{\mathcal{L}} = \mathcal{L}_{\text{task}} + \text{regularization terms}$, and the expectation is over both the data distribution and the GARN noise.

The proof proceeds in three steps:

1. Gradient mixing preserves the descent direction (Lemma 7.1 guarantees positive-definite mixing).
2. GARN prevents convergence to degenerate routing fixed points (Theorem 8.1).
3. The Lagrangian budget term is a linear constraint; its dual variable converges by Theorem 6.1.

10.3 Operator-Theoretic View: Core + LoRA as Perturbation

The Core + LoRA factorization can be formalized more precisely using Kato's perturbation theory for bounded linear operators on Hilbert spaces.

Let $\mathcal{H} = \mathbb{R}^d$ be the representation space and $\mathcal{B}(\mathcal{H})$ the space of bounded linear operators on $\mathcal{H}$. The expert operator $T_{g,f,k} \in \mathcal{B}(\mathcal{H})$ acts as:

$$T_{g,f,k}(\mathbf{x}) = \sigma\left(\mathbf{x}(\mathbf{W}_g^{\text{core}} + \mathbf{A}^{(g,f,k)}\mathbf{B}^{(g,f,k)})\right)\mathbf{W}_g^{\text{core},\downarrow}, \tag{38}$$

where σ is the activation function and $\mathbf{W}_g^{\text{core},\downarrow}$ is the shared down-projection.

Proposition 10.2 (Perturbation Bound on Linear Part)**.** Let T_g^{core} be the Core linear operator (LoRA terms set to zero, *before* activation σ) and $\Delta T_{g,f,k}$ the LoRA perturbation to the linear part. Then the operator norm of the linear perturbation satisfies:

$$\|\mathbf{W}_g^{\text{core}} + \mathbf{A}^{(g,f,k)}\mathbf{B}^{(g,f,k)} - \mathbf{W}_g^{\text{core}}\|_{\text{op}} = \|\mathbf{A}^{(g,f,k)}\mathbf{B}^{(g,f,k)}\|_{\text{op}} \le \|\mathbf{A}^{(g,f,k)}\|_2 \cdot \|\mathbf{B}^{(g,f,k)}\|_2 \cdot \frac{\alpha}{r}. \tag{39}$$

For the full nonlinear expert operator $T_{g,f,k}$ with activation σ (Lipschitz constant L_σ), the Lipschitz constant of the perturbation satisfies:

$$\text{Lip}(T_{g,f,k} - T_g^{\text{core}}) \le L_\sigma \cdot \|\mathbf{W}_g^{\text{core},\downarrow}\|_{\text{op}} \cdot \|\mathbf{A}^{(g,f,k)}\|_2 \cdot \|\mathbf{B}^{(g,f,k)}\|_2 \cdot \frac{\alpha}{r}. \tag{40}$$

Since $\text{rank}(\mathbf{AB}) = r \ll d$, the perturbation is confined to a low-dimensional subspace, and its spectral impact decays as $O(\alpha/r)$.

Remark 10.1. The bound above applies to the Lipschitz constant of the operator difference, not to the operator norm of the difference of nonlinear operators. A direct bound on $\|T_{g,f,k} - T_g^{\text{core}}\|_{\text{op}}$ would require additional assumptions on the input distribution (e.g., bounded $\|\mathbf{x}\|$). The Lipschitz bound is sufficient to guarantee that the LoRA adaptation does not drastically alter the input-output mapping of the expert.

11 Ontogenic Pre-training and Category-Theoretic Bridge

11.1 From "Zero Quarks" to Language

A distinctive feature of AS-MoE is the **ontogenic pre-training** protocol, which begins not with language data, but with synthetic "quark-level" attributes—elementary properties that compose to form the representational primitives of language. The motivation is two-fold:

1. **Structural bootstrapping**: By first training the expert ensemble on well-structured synthetic data, we establish a functional routing topology (which experts specialize in which attribute combinations) before exposing the model to the noise and ambiguity of natural language.

2. **Compositional transfer**: The attributes are designed to compose hierarchically, mirroring the compositional structure of language (phonemes → morphemes → words → phrases). This compositional prior is formally captured by the category-theoretic framework below.

11.2 Phase 0: Synthetic Quarks

In Phase 0, the model is trained on synthetic data where each input is a bag of discrete attributes ("quarks"):

$$\mathbf{x}_{\text{quark}} = \text{Embed}(a_1) \oplus \text{Embed}(a_2) \oplus \cdots \oplus \text{Embed}(a_m), \tag{41}$$

where $a_i \in \mathcal{A}$ are attributes from a predefined vocabulary $\mathcal{A}$, and $\oplus$ denotes summation-based aggregation: $\oplus_i \text{Embed}(a_i) = \frac{1}{m}\sum_{i=1}^{m} \text{Embed}(a_i)$. (Alternative aggregation strategies such as attention-weighted combination are discussed in Section 16.)

The training objective in Phase 0 is attribute prediction:

$$\mathcal{L}_{\text{Phase 0}} = -\sum_{i=1}^{m} \log P(a_i \mid \mathbf{x}_{\text{quark}}). \tag{42}$$

This objective forces the routing topology to organize around attribute-level structure: specific flavors specialize in specific attribute types, and the generation hierarchy reflects attribute complexity.

11.3 Phase 1: Language Immersion

In Phase 1, the model transitions to language data. The Core weights from Phase 0 are frozen, and only the LoRA adapters and routers are trained. An **inductive bias loss** encourages the Phase 1 routing to be consistent with the Phase 0 attribute structure:

$$\mathcal{L}_{\text{IB}} = D_{\text{KL}}\left(\pi_{\text{lang}} \parallel \pi_{\text{quark}}^{(\text{frozen})}\right), \tag{43}$$

where $\pi_{\text{quark}}^{(\text{frozen})}$ is the routing distribution from Phase 0 (used as a fixed prior) and π_{lang} is the Phase 1 routing distribution.

11.4 Category-Theoretic Formalization

The compositional bridge from quarks to language can be formalized as a **functor** between categories.

Definition 11.1 (Quark Category $\mathbf{C}_{\text{quark}}$)**.** The quark category has:

- **Objects**: Finite multisets of attributes $[a_1, a_2, \ldots, a_m]$, where $a_i \in \mathcal{A}$.
- **Morphisms**: Attribute-preserving maps $f : [a_i] \to [b_j]$ such that $[b_j]$ can be obtained from $[a_i]$ by adding, removing, or substituting attributes.

$\mathbf{C}_{\text{quark}}$ has finite coproducts (disjoint union of attribute multisets).

Definition 11.2 (Language Category $\mathbf{C}_{\text{lang}}$)**.** The language category has:

- **Objects**: Token sequences $[t_1, t_2, \ldots, t_n]$, where $t_i \in \mathcal{V}$ (vocabulary).
- **Morphisms**: Context-preserving maps (e.g., synonym substitution, paraphrase).

$\mathbf{C}_{\text{lang}}$ has finite coproducts (concatenation of token sequences).

Theorem 11.1 (Compositional Bridge)**.** There exists a functor $\mathcal{F} : \mathbf{C}_{\text{quark}} \to \mathbf{C}_{\text{lang}}$ that:

1. Preserves coproducts: $\mathcal{F}(A \sqcup B) \cong \mathcal{F}(A) \sqcup \mathcal{F}(B)$.
2. Maps each attribute $a \in \mathcal{A}$ to a distribution over token sequences in $\mathcal{V}$.
3. Preserves the compositionality of attributes: the representation of a composite quark is determined by the representations of its constituent attributes.

The inductive bias loss $\mathcal{L}_{\text{IB}}$ is the practical instantiation of the functor constraint: it ensures that the routing topology learned in Phase 0 (over quarks) is approximately preserved in Phase 1 (over language).

The functorial perspective provides a formal justification for why ontogenic pre-training should improve sample efficiency: the functor $\mathcal{F}$ is learned in Phase 0 from the well-structured quark data, and Phase 1 only needs to refine $\mathcal{F}$ on the noisier language data rather than learning it from scratch. The coproduct preservation property ensures that compositional structure is maintained across the bridge.

12 Systems Design: CUDA and Data-Oriented Design

12.1 Data-Oriented Design Principles

The AS-MoE architecture is designed from the ground up for efficient GPU execution, following Data-Oriented Design (DoD) principles. The key insight is that the Generation/Flavor/Channel topology maps naturally to a **Structure of Arrays (SoA)** memory layout, where all parameters of the same type are stored contiguously regardless of which expert they belong to.

The SoA layout enables:

1. **Coalesced memory access**: All reads of the same parameter type (e.g., all Core up-weights) are contiguous in memory, maximizing GPU memory bandwidth utilization.
2. **Batched operations**: Grouped GEMM can process all experts of the same generation in a single kernel launch.
3. **Efficient all-to-all**: In distributed training, the SoA layout simplifies the all-to-all communication pattern: each GPU sends/receives contiguous slices of the parameter arrays.

Table 1: Memory layout comparison: AoS vs. SoA for AS-MoE parameters.

Parameter	**AoS (traditional)**	**SoA (DoD)**
Core up-weights	Scattered per expert	$\mathbf{W}_{\text{up}}^{\text{core}} \in \mathbb{R}^{G\times d\times d_{\text{ff}}^{(\max)}}$ (contiguous)
LoRA **A** matrices	Scattered per expert	$\mathbf{A} \in \mathbb{R}^{G\times F\times K\times d\times r}$ (contiguous)
LoRA **B** matrices	Scattered per expert	$\mathbf{B} \in \mathbb{R}^{G\times F\times K\times r\times d'}$ (contiguous)
Router weights	Per-layer, per-type	$\mathbf{W}_g \in \mathbb{R}^{L\times d\times G}$, $\mathbf{W}_f \in \mathbb{R}^{L\times K\times d\times F}$ (contiguous)

12.2 Fused Grouped GEMM

The key computational kernel in AS-MoE is the **Grouped GEMM**, which performs batched matrix multiplication across expert groups with varying dimensions. Unlike standard batched GEMM (which requires uniform dimensions), Grouped GEMM handles the heterogeneous $d_{\text{ff}}^{(g)}$ across generations:

```python
# Conceptual Fused Grouped GEMM kernel
# All generations processed in a single kernel launch
# SoA layout: weights[generation] has shape
#   (num_experts_in_gen, d_model, d_ff_gen)
def fused_grouped_gemm(
    x,            # (tokens, d_model)
    weights,      # SoA: list of (experts, d, d_ff_g)
    route_map,    # (tokens,) -> (generation, flavor, channel)
    active_mask   # (tokens, experts) sparse
):
    outputs = []
    for g in range(G):
        # Gather tokens routed to generation g
        token_idx = (route_map[:, 0] == g)
        x_g = x[token_idx]  # (tokens_g, d_model)
        # Batched GEMM for all (f,k) in generation g
        # All experts in same generation share d_ff
        out_g = grouped_gemm(
            x_g,
            weights[g],        # (F*K, d, d_ff_g)
            active_mask[token_idx, g*F*K:(g+1)*F*K]
        )
        outputs.append(out_g)
    return scatter_combine(outputs, route_map)
```

The fusion eliminates kernel launch overhead (which can dominate runtime for small expert groups) and enables shared memory reuse across the batched GEMM operations. On NVIDIA H100 GPUs, fused Grouped GEMM achieves >90% of peak compute utilization for expert groups of ≥ 8 experts.

12.3 Persistent Kernels: FlashDMoE Integration

For large-scale distributed training, AS-MoE leverages the **FlashDMoE** persistent kernel design. Traditional MoE implementations launch separate kernels for:

1. Dispatch (all-to-all send + local token rearrangement).
2. Expert computation (GEMM + activation).
3. Combine (all-to-all receive + output assembly).

Each kernel launch incurs overhead ($\sim$5–10μs on H100), which accumulates across the L MoE layers in a forward pass. FlashDMoE's persistent kernel design keeps a single kernel resident on the GPU, processing all three phases without re-launching:

$$\text{Latency}_{\text{persistent}} \approx \frac{\text{FLOPs}}{\text{Peak TFLOP/s}} + T_{\text{comm}}, \tag{44}$$

whereas the traditional approach adds $3L \times T_{\text{launch}}$ to the critical path.

12.4 Fused Computation-Communication via NVSHMEM

For multi-node training, AS-MoE employs **NVSHMEM** (NVIDIA Shared Memory) to overlap computation and communication. The key technique is *expert prefetching*: while the current layer's experts are computing, the next layer's expert weights are asynchronously fetched from remote GPUs.

```python
# Fused compute-communicate pattern
def fused_layer_forward(x, layer_idx, nvshmem_handle):
    # Launch async fetch for next layer's weights
    if layer_idx < L - 1:
        nvshmem_handle.prefetch(
            weights[layer_idx + 1],
            stream=cuda_stream_comm)

    # Compute current layer on default stream
    output = asmoe_layer_forward(x, layer_idx)

    # Synchronize: ensure prefetch is complete
    if layer_idx < L - 1:
        cuda_stream_comm.synchronize()

    return output
```

NVSHMEM provides remote direct memory access (RDMA) between GPUs, bypassing the CPU and reducing communication latency by 30–50% compared to NCCL-based all-to-all. The prefetching pattern hides communication latency behind computation, achieving near-zero communication overhead when compute time exceeds communication time (which holds for typical expert dimensions).

13 Training Methodology

13.1 Three-Phase Training Protocol

AS-MoE training proceeds in three phases:

13.1.1 Phase 0: Ontogenic Pre-training (Quark-Level)

1. Generate synthetic attribute data (Section 11).
2. Train the full AS-MoE model on attribute prediction $\mathcal{L}_{\text{Phase 0}}$.
3. Use Expert Choice routing for maximum load balance.
4. Train until routing topology stabilizes (measured by entropy of per-flavor assignment convergence).
5. **Outcome**: A stable routing topology where flavors correspond to attribute types and generations correspond to attribute complexity.

13.1.2 Phase 1: Language Immersion

1. Freeze Core weights from Phase 0.
2. Switch to language data with standard LM objective.

3. Maintain inductive bias loss $\mathcal{L}_{\text{IB}}$ (Section 11).
4. Transition from Expert Choice to Token Choice routing with a warmup schedule.
5. Enable GARN noise injection.
6. Enable gradient mixing.
7. **Outcome**: A language-capable model whose routing topology is grounded in compositional attribute structure.

13.1.3 Phase 2: Domain Transfer (Optional)

1. Freeze Core weights.
2. Re-initialize LoRA adapters for the target domain.
3. Re-initialize routers (allowing new routing patterns).
4. Apply EWC++ to Shared Expert (see below).
5. Apply routing replay buffer (see below).
6. **Outcome**: A domain-specialized model that retains the compositional routing structure from pre-training.

13.2 Anti-Forgetting: Activation-Importance Consolidation (AIC)

Standard Elastic Weight Consolidation (EWC) prevents catastrophic forgetting by penalizing changes to parameters deemed important for the source domain. AS-MoE replaces EWC with **Activation-Importance Consolidation (AIC)**, which measures parameter importance via the sensitivity of *activations* rather than the classical Fisher information on parameters, and applies it selectively to the Shared Expert (g=0):

Definition 13.1 (Activation-Importance Consolidation)**.** The AIC regularization loss is:

$$\mathcal{L}_{\text{AIC}} = \sum_{i \in \mathcal{I}_{g=0}} I_i^{(\text{act})} \left(\theta_i - \theta_i^{\text{pre}}\right)^2, \tag{45}$$

where $\mathcal{I}_{g=0}$ is the set of Shared Expert parameter indices, θ_i^{pre} are the pre-trained values, and $I_i^{(\text{act})}$ is the *activation importance* of parameter θ_i:

$$I_i^{(\text{act})} = \mathbb{E}_{\mathbf{x} \sim \mathcal{D}_{\text{source}}} \left[\left(\frac{\partial \log p(\mathbf{x}|\boldsymbol{\theta})}{\partial a_i} \right)^2 \right], \tag{46}$$

where a_i is the activation (pre- or post-nonlinearity) corresponding to parameter θ_i. Unlike the diagonal Fisher information $F_i = \mathbb{E}[(\partial \log p / \partial \theta_i)^2]$, which measures sensitivity of the log-likelihood to parameter changes, $I_i^{(\text{act})}$ measures how strongly the log-likelihood depends on the *activation* modulated by θ_i, thereby capturing the context-dependent importance of θ_i in MoE settings where the same parameter has different effective importance depending on which experts are active.

The selective application to the Shared Expert (g=0) is justified by Proposition 3.1: the Shared Expert processes every token, making it the most critical component for retaining source-domain knowledge. Routed experts (higher generations) are expected to adapt to the new domain, so constraining them is counterproductive.

13.3 Routing Replay Buffer

To prevent the router from "forgetting" how to route source-domain tokens during transfer, AS-MoE maintains a **Routing Replay Buffer** that stores the routing statistics of the source domain:

Definition 13.2 (Routing Replay Buffer)**.** The replay buffer stores:

$$\mathcal{B}_{\text{replay}} = \left\{ \left(\boldsymbol{\mu}_\pi^{(\ell)}, \boldsymbol{\Sigma}_\pi^{(\ell)} \right) \right\}_{\ell=1}^{L}, \tag{47}$$

where $\boldsymbol{\mu}_\pi^{(\ell)} \in \mathbb{R}^{N_e}$ is the mean routing distribution at layer ℓ (averaged over source-domain tokens) and $\boldsymbol{\Sigma}_\pi^{(\ell)} \in \mathbb{R}^{N_e \times N_e}$ is the covariance of the routing distribution. During transfer training, the replay loss is:

$$\mathcal{L}_{\text{replay}} = \sum_{\ell=1}^{L} \left[\|\boldsymbol{\mu}_\pi^{(\ell,t)} - \boldsymbol{\mu}_\pi^{(\ell,\text{src})}\|^2 + \lambda_\Sigma \|\boldsymbol{\Sigma}_\pi^{(\ell,t)} - \boldsymbol{\Sigma}_\pi^{(\ell,\text{src})}\|_F^2 \right]. \tag{48}$$

The covariance term is essential: matching only the mean routing distribution would allow the router to concentrate all variance on a single expert (low entropy), which the covariance term prevents. The replay buffer is lightweight, storing only $O(L \cdot N_e^2)$ values, which is negligible compared to the model parameters.

14 Reference Implementation

14.1 GradientMixing (Dimension-Preserving, Per-Channel)

The gradient mixing autograd function preserves full tensor dimensions in the backward pass, applying the correlation-based mixing via `einsum`. As established in Remark 7.1, mixing is applied independently within each channel over the $N = G \times F$ experts of that channel:

```python
class GradientMixing(Function):
    @staticmethod
    def forward(ctx, expert_stack, weights, gamma):
        # expert_stack: [N, B, T, D] - stacked expert
        #   outputs within ONE channel (N = G*F)
        # weights: [B, T, N] - routing weights
        # gamma: mixing strength
        ctx.save_for_backward(
            expert_stack.bfloat16(),
            weights.bfloat16(),
            torch.tensor(gamma))
        ctx.gamma = gamma
        # Weighted sum: [B, T, D]
        return torch.einsum('bti,ibtd->btd', weights,
                            expert_stack.float())

    @staticmethod
    def backward(ctx, grad_output):
        # grad_output: [B, T, D]
        expert_stack, weights, gamma_t = ctx.saved_tensors
        gamma = gamma_t.item()
        N = expert_stack.shape[0]

        # Compute correlation matrix R: [N, N]
        flat = expert_stack.float().reshape(N, -1)  # [N, B*T*D]
        R = torch.nn.functional.cosine_similarity(
            flat.unsqueeze(1), flat.unsqueeze(0), dim=-1)
        M = torch.eye(N, device=grad_output.device) + gamma * R

        # Raw gradients: [N, B, T, D]
        raw = torch.einsum('btd,bti->ibtd',
                           grad_output, weights.float())
```

```python
        # Mixed gradients via einsum (preserves dims):
        # mixed_i = sum_j M[j,i] * raw_j
        mixed = torch.einsum('ji,jbtd->ibtd', M, raw)

        return mixed, None, None
```

14.2 ASMoELayer (Vectorized Forward)

The following implementation eliminates Python loops over batch and sequence dimensions, using vectorized operations and batched GEMM:

```python
class ASMoELayer(nn.Module):
    def __init__(
        self, d_model, G=3, F=4, K=2,
        d_ffs=[1024, 2048, 4096],
        budgets=[1.0, 2.0, 4.0],
        lora_r=8, lora_alpha=16.0,
        gamma_mix=0.1, target_budget=2.0,
        beta_ema=0.99, eta_mu=0.01,
        top_m=1, d_proj=None,
        garn_sigma_base=0.1, garn_eta=2.0,
        garn_alpha=0.5, gen_temp=1.0
    ):
        super().__init__()
        self.G, self.F, self.K = G, F, K
        self.gamma_mix = gamma_mix
        self.target_budget = target_budget
        self.budgets_val = budgets
        self.d_ffs = d_ffs
        self.lora_r = lora_r
        self.lora_alpha = lora_alpha
        self.top_m = top_m
        self.garn_sigma_base = garn_sigma_base
        self.garn_eta = garn_eta
        self.garn_alpha = garn_alpha
        d_proj = d_proj or d_model // 2

        # Channel projections (multi-view)
        self.projs = nn.ParameterList([
            nn.Parameter(torch.randn(d_model, d_proj)
                         * (2.0/d_model)**0.5)
            for _ in range(K)])

        # Generation routers (one per channel, consensus)
        self.gen_routers = nn.ModuleList([
            nn.Linear(d_proj, G) for _ in range(K)])

        # Flavor routers: per (generation, channel)
        # W_f^{(g,k)} as in Definition 5.2
        self.flavor_routers = nn.ModuleDict({
            f"{g}_{k}": nn.Linear(d_proj, F)
            for g in range(G) for k in range(K)})

        # Expert params: Core + LoRA (same as before)
        self.cores_up = nn.ParameterList()
        self.cores_down = nn.ParameterList()
        self.lora_A_up = nn.ParameterList()
        self.lora_B_up = nn.ParameterList()
        self.lora_A_down = nn.ParameterList()
        self.lora_B_down = nn.ParameterList()
        for g in range(G):
            d_ff = d_ffs[g]
            self.cores_up.append(
                nn.Parameter(torch.randn(d_model, d_ff)
                             * (2.0/d_model)**0.5))
            self.cores_down.append(
                nn.Parameter(torch.randn(d_ff, d_model)
                             * (2.0/d_ff)**0.5))
            for f in range(F):
                for k in range(K):
                    self.lora_A_up.append(
```

```
                        nn.Parameter(torch.randn(
                            d_model, lora_r)
                            * (2.0/d_model)**0.5))
                    self.lora_B_up.append(
                        nn.Parameter(torch.zeros(
                            lora_r, d_ff)))
                    self.lora_A_down.append(
                        nn.Parameter(torch.randn(
                            d_ff, lora_r)
                            * (2.0/d_ff)**0.5))
                    self.lora_B_down.append(
                        nn.Parameter(torch.zeros(
                            lora_r, d_model)))

        # Budget controller: separate EMA and dual var
        self.register_buffer(
            'budget_ema', torch.tensor(0.0))
        self.register_buffer(
            'mu', torch.tensor(0.0))
        self.beta_ema = beta_ema
        self.eta_mu = eta_mu
        self.gen_temp = gen_temp

    def _lora_idx(self, g, f, k):
        return g * self.F * self.K + f * self.K + k

    def _garn_noise(self, logits, probs):
        """GARN: entropy-adaptive + gradient-adaptive noise.
        NOTE: The gradient-adaptive term uses |logits| as a
        proxy for |grad_z L|, since true logit gradients are
        unavailable in forward pass. This is justified by the
        correlation between logit magnitude and softmax-gradient
        magnitude: d L/d z_i = pi_i (delta_{i,y} - pi_y),
        which is proportional to pi_i ~ exp(z_i) for the
        dominant expert. For a true gradient-adaptive term,
        a two-pass forward (first pass without noise, then
        backward to obtain grad_z, then second pass with
        gradient-scaled noise) would be needed at 2x cost.
        """
        H = -(probs * (probs + 1e-8).log()).sum(-1)
        sigma = self.garn_sigma_base * torch.exp(
            -self.garn_eta * H)
        # Gradient-adaptive term (proxy: logit magnitude
        # correlates with |grad_z L| via softmax Jacobian)
        sigma = sigma.unsqueeze(-1) * (
            1 + self.garn_alpha * logits.abs())
        return sigma * torch.randn_like(logits)

    def forward(self, x):
        B, T, D = x.shape
        ch_outputs = []
        gen_dists = []  # for consensus loss
        flv_dists = []  # for diversity loss

        for k in range(self.K):
            h = x @ self.projs[k]  # (B, T, d_proj)

            # --- Generation routing with GARN ---
            gen_logits = self.gen_routers[k](h)
            if self.training:
                gen_logits = gen_logits + self._garn_noise(
                    gen_logits,
                    F.softmax(gen_logits / self.gen_temp,
                              dim=-1))
            gen_probs = F.softmax(
                gen_logits / self.gen_temp, dim=-1)

            # STE: hard forward, soft backward
            g_hard = F.one_hot(
                gen_probs.argmax(-1), self.G
            ).float()
            g_hard = (g_hard
                      + gen_probs - gen_probs.detach())
```

```python

            gen_dists.append(gen_probs)

            # --- Flavor routing with GARN ---
            ch_expert_outs = []
            ch_weights = []
            for g in range(self.G):
                flv_logits = self.flavor_routers[
                    f"{g}_{k}"](h)
                if self.training:
                    flv_logits = flv_logits + self._garn_noise(
                        flv_logits,
                        F.softmax(flv_logits, dim=-1))
                flv_probs = F.softmax(flv_logits, dim=-1)
                flv_dists.append(flv_probs)

                # Top-m flavor selection
                top_vals, top_idx = flv_probs.topk(
                    self.top_m, dim=-1)
                # One-hot for selected flavors
                flv_mask = F.one_hot(
                    top_idx, self.F
                ).float().sum(dim=-2)  # (B, T, F)
                flv_mask = (flv_mask
                            + flv_probs - flv_probs.detach())

                # Gather tokens for this generation
                g_mask = (gen_probs.argmax(-1) >= g
                          ).float()  # curriculum
                tokens_g = x  # process all, mask later

                for f in range(self.F):
                    idx = self._lora_idx(g, f, k)
                    scale = self.lora_alpha / self.lora_r
                    W_up = (self.cores_up[g]
                            + scale * self.lora_A_up[idx]
                            @ self.lora_B_up[idx])
                    W_down = (self.cores_down[g]
                              + scale * self.lora_A_down[idx]
                              @ self.lora_B_down[idx])
                    hidden = F.gelu(tokens_g @ W_up)
                    out_f = hidden @ W_down  # (B, T, D)
                    ch_expert_outs.append(out_f)
                    # Weight = flv_mask for f * g_mask
                    w = (flv_mask[..., f:f+1]
                         * g_mask.unsqueeze(-1))
                    ch_weights.append(w)

            # Stack and apply gradient mixing
            expert_stack = torch.stack(
                ch_expert_outs, dim=0)  # [N, B, T, D]
            weight_stack = torch.stack(
                ch_weights, dim=-1)      # [B, T, N]
            if self.training:
                ch_out = GradientMixing.apply(
                    expert_stack, weight_stack,
                    self.gamma_mix)
            else:
                ch_out = torch.einsum(
                    'bti,ibtd->btd',
                    weight_stack, expert_stack)
            ch_outputs.append(ch_out)

        # Average over channels (no curriculum norm)
        final = sum(ch_outputs) / self.K

        # --- Budget: actual FLOPs from hard routing ---
        # Curriculum cost: a token assigned to generation g
        # receives contributions from ALL generations <= g.
        # So actual_cost = sum_g P(g_assign >= g) * B^(g),
        # which correctly accounts for the cumulative budget.
        g_assign = gen_dists[0].argmax(-1)  # (B, T)
        actual_cost = sum(
```

```
            (g_assign >= g).float().mean()
            * self.budgets_val[g]
            for g in range(self.G))

        # Two-phase budget update (Lagrangian dual ascent):
        # Phase A: EMA for variance-reduced budget estimate
        self.budget_ema.copy_(
            self.beta_ema * self.budget_ema
            + (1 - self.beta_ema) * actual_cost)
        # Phase B: gradient step on mu (using PRE-update mu
        # for budget_loss, then update mu for next step).
        # NOTE: budget_loss uses old mu; mu is updated
        # simultaneously with theta. This creates a one-step
        # lag, which is acceptable in practice but differs
        # from strict primal-dual (which updates mu AFTER
        # the theta gradient step). For strict primal-dual,
        # move mu.update() outside the forward pass.
        budget_loss = self.mu * (
            self.budget_ema - self.target_budget)
        self.mu.copy_(torch.clamp(
            self.mu + self.eta_mu * (
                self.budget_ema - self.target_budget),
            min=0.0))

        # --- Consensus loss (cross-channel) ---
        cons_loss = -sum(
            F.cosine_similarity(
                gen_dists[i], gen_dists[j], dim=-1
            ).mean()
            for i in range(self.K)
            for j in range(i+1, self.K))

        # --- Diversity loss (per-generation averaging) ---
        # Definition 5.2: div is averaged per (g,k) pair,
        # not across all generations simultaneously.
        div_loss = sum(
            F.mse_loss(
                flv.mean(dim=[0,1]),  # avg over B,T
                torch.ones(self.F, device=x.device) / self.F)
            for flv in flv_dists
        ) / len(flv_dists)

        # --- Channel orthogonality loss ---
        ortho_loss = sum(
            torch.norm(
                self.projs[i].T @ self.projs[j], p='fro')**2
            for i in range(self.K)
            for j in range(i+1, self.K))

        return final, {
            'budget': budget_loss,
            'consensus': cons_loss,
            'diversity': div_loss,
            'ortho': ortho_loss,
        }
```

14.3 Routing Replay Buffer

```
class RoutingReplayBuffer:
    """Stores mean and covariance of routing distributions
    from source domain for anti-forgetting during transfer."""

    def __init__(self, num_layers, num_experts,
                 device='cuda'):
        self.num_layers = num_layers
        self.num_experts = num_experts
        self.device = device
        # Store on GPU for in-place lerp_ updates
        self.means = [torch.zeros(num_experts,
                      device=device)
                      for _ in range(num_layers)]
```

```python
        self.covs = [torch.eye(num_experts,
                     device=device)
                     for _ in range(num_layers)]
        self.count = 0

    @torch.no_grad()
    def update(self, routing_distributions):
        """Update buffer with new routing distributions.
        Args:
            routing_distributions: list of (B*T, N_e) tensors,
                one per layer.
        """
        self.count += 1
        alpha = min(0.1, 1.0 / self.count)
        for l, pi in enumerate(routing_distributions):
            mean_new = pi.mean(dim=0)
            cov_new = torch.cov(pi.T)
            # In-place EMA update (avoids CPU<->GPU copies)
            self.means[l].lerp_(mean_new, alpha)
            self.covs[l].lerp_(cov_new, alpha)

    def compute_loss(self, current_distributions, lam_sigma=0.1):
        """Compute replay loss against stored statistics."""
        loss = 0.0
        for l, pi in enumerate(current_distributions):
            mean_curr = pi.mean(dim=0)
            cov_curr = torch.cov(pi.T)
            loss += F.mse_loss(mean_curr, self.means[l])
            loss += lam_sigma * F.mse_loss(
                cov_curr, self.covs[l])
        return loss / self.num_layers
```

15 Experimental Design

15.1 Proposed Benchmarks

We propose the following experimental protocol to validate AS-MoE:

Table 2: Proposed experimental configurations for AS-MoE validation.

Config	Params	G	F	K
AS-MoE-S	1.3B	2	4	2
AS-MoE-M	6.7B	3	4	2
AS-MoE-L	13B	3	8	4

15.2 Ablation Studies

The following ablation studies are proposed to isolate the contribution of each architectural component:

1. **Generation hierarchy**: Remove the curriculum logic (only use the selected generation, not all $\leq g$).
2. **Gradient mixing**: Set $\gamma = 0$ (no mixing) and compare training stability and expert utilization.
3. **GARN**: Replace GARN with fixed Gaussian noise ($\sigma = \text{const}$) and measure router collapse rates.
4. **Path consensus**: Set $\lambda_{\text{path}} = 0$ and measure cross-layer routing consistency.

5. **Ontogenic pre-training**: Train from scratch on language data (skip Phase 0) and compare sample efficiency.

6. **Core + LoRA**: Replace with full-rank expert weights (no factorization) and compare transfer performance.

7. **Hybrid routing**: Use only Token Choice throughout training and compare with the EC → TC transition.

16 Discussion

16.1 The Quark Analogy: From Metaphor to Method

The development of AS-MoE illustrates a broader pattern in architecture design: the transition from informal analogy to formal theory. The original quark model analogy—with its "generations" (up/down, strange/charm, top/bottom), "flavors" (functional specialization), and "colors" (multi-view representation)—provided an intuitive organizational framework. However, each component has been independently justified through engineering necessity:

- **Generation hierarchy** is justified by the observation that not all tokens require the same computational depth (a fact established by early-exit and adaptive computation research).
- **Flavor specialization** is justified by the success of shared experts (DeepSeek) and the failure mode of unstructured routing (expert redundancy).
- **Multi-view channels** are justified by the multi-head attention principle: independent projections capture complementary aspects of the input.
- **Confinement** (experts must cooperate, not isolate) is justified by the empirical observation of mode collapse in MoE training and the theoretical analysis via gradient mixing (Lemma 7.1).

The quark analogy thus served as a productive metaphor that guided the design space exploration, but the final architecture is justified on its own terms.

16.2 Known Implementation Issues

The reference implementation (Section 14) addresses the major code-text discrepancies identified during review, but several issues remain:

1. **Expert Choice → Token Choice transition**: The hybrid routing strategy (Section 5) requires a distillation loss to smooth the EC→TC transition. This loss is not yet implemented and is deferred to the Triton kernel implementation.

2. **Reference implementation is not truly sparse**: The current forward pass (Section 14) processes *all* tokens through *all* experts and then masks outputs by routing weights. This means the FLOP cost is $O(B \cdot T \cdot G \cdot F \cdot K \cdot D \cdot d_{\text{ff}})$—full dense compute—rather than the sparse $O(B \cdot T \cdot N_{\text{active}} \cdot D \cdot d_{\text{ff}})$ described in the architecture. The reference implementation demonstrates *correctness* of the routing and mixing logic, but **not efficiency**. True sparse execution requires custom CUDA/Triton kernels with block scheduling that only launch expert computation for routed tokens.

3. **GARN gradient-adaptive term uses logit proxy**: As discussed in Remark 8.1, the gradient term $|\nabla_{z_i}\mathcal{L}|$ in Definition 8.1 is approximated by $|z_i|$ in the reference implementation,

since true logit gradients are unavailable during the forward pass. This approximation preserves qualitative behavior but introduces bias. A two-pass GARN would be more accurate at 2× cost.

4. **Full vectorization requires custom CUDA**: The current implementation eliminates Python loops over batch/sequence but still iterates over experts within a channel. A fully fused implementation requires custom CUDA/Triton kernels with block scheduling (see Section 12).

5. **SoA layout not yet realized in PyTorch**: The SoA memory layout described in Section 12 is conceptual. The PyTorch reference uses standard `nn.ParameterList` (AoS). Production deployment requires explicit SoA tensors with expert offsets per generation:

```
# SoA layout for production
self.w_up = nn.Parameter(torch.empty(
    G, F, K, d_model, d_ff_max))
# + expert_offsets for varying d_ff per g
```

6. **Gradient mixing across DDP**: The all-gather for global correlation matrix computation is described but not implemented. This requires a custom communication hook in PyTorch DDP.

7. **Simultaneous vs. sequential dual update**: The budget controller updates μ simultaneously with the forward pass, creating a one-step lag between budget violation and penalty (see Section 14 for the detailed comment). Strict primal-dual methods update μ *after* the primal gradient step. This is acceptable in practice but theoretically imprecise.

16.3 Limitations and Open Questions

Several open questions remain:

1. **Scalability**: The current analysis assumes a fixed number of experts per layer. Scaling to millions of experts (as in some proposed architectures) requires different routing mechanisms and systems design.

2. **Dynamic G, F, K**: The current architecture uses fixed topology hyperparameters. Adaptive topology—where the number of active flavors or channels varies per layer or during training—is an unexplored direction.

3. **Optimal LoRA rank**: The choice of LoRA rank r is currently a hyperparameter. Theory from Kato perturbation analysis (Proposition 10.2) suggests that r should scale with the spectral gap of the Core operator, but this connection needs formalization. A concrete conjecture: if δ is the spectral gap of $\mathbf{W}^{\text{core}}$ (distance from the r-th to $(r{+}1)$-th singular value), then LoRA rank r satisfying $r < \delta/2$ ensures that the perturbation does not cause spectral bifurcation, by the Gershgorin circle theorem applied to the Kato bound.

4. **Functor approximation**: The functor $\mathcal{F}$ (Theorem 11.1) is approximated in practice by the inductive bias loss. Characterizing the approximation error and its impact on downstream performance is an open theoretical question. A starting point is to estimate the sample complexity of ontogenic pre-training: how many synthetic quark samples are needed for ϵ-approximation of $\mathcal{F}$, using VC-dimension or Rademacher complexity of the categorical morphisms?

5. **Triton kernel optimization**: The current implementation uses PyTorch with conceptual Grouped GEMM. A production implementation requires custom Triton kernels with block scheduling and GARN integration, which is deferred to future work.

6. **Aggregation strategy for ontogenic embeddings**: The quark aggregation operator $\oplus$ (Section 11) currently uses mean pooling. Attention-weighted aggregation may better capture attribute interactions but introduces additional parameters.

7. **Ablation scale equivalence**: In the proposed ablation studies (Section 15), AS-MoE-S (1.3B total parameters) should be compared against dense models with equivalent *active* parameter counts, not equivalent *total* parameter counts. The equivalent dense baseline for AS-MoE-S with target budget 2.0 is approximately 0.8B parameters.

8. **Communication lower bound for fused computation-communication**: For the fused compute-communicate pattern (Section 12), we conjecture that latency $\geq \max(T_{\text{comp}}, T_{\text{comm}})$ and that prefetching achieves this lower bound when $T_{\text{comp}} \geq T_{\text{comm}}$. A formal proof would establish the optimality of the NVSHMEM-based pipeline.

17 Conclusion

We have presented AS-MoE, an architecture that imposes principled topological, algebraic, and dynamical structure on Mixture of Experts. By organizing experts along three orthogonal axes (Generation, Flavor, Channel), factorizing parameters into Core + LoRA components with Kato perturbation-theoretic justification, enforcing FLOP budgets via Lagrangian duality with two-phase dual updates, preventing expert isolation through covariance gradient mixing with Gershgorin convergence guarantees, stabilizing routing via GARN, ensuring cross-layer coherence via Path Consensus, and formalizing ontogenic pre-training through category-theoretic functors, AS-MoE provides a comprehensive, formally grounded framework for structured MoE design. The systems design leverages Data-Oriented Design principles, fused Grouped GEMM, persistent kernels, and NVSHMEM for efficient GPU execution. We believe this work represents a step toward MoE architectures that are not only scalable and efficient, but also theoretically principled and predictably behaved.

A Engineering PRD: AS-MoE Production System

This appendix provides a Product Requirements Document (PRD) for a production AS-MoE system, translating the architectural theory into engineering specifications.

A.1 Overview

The AS-MoE Production System is a distributed training and inference framework for the Adaptive Structured Mixture of Experts architecture. It supports the full training lifecycle: ontogenic pre-training (Phase 0), language immersion (Phase 1), and domain transfer (Phase 2), with production inference serving.

A.2 System Requirements

A.3 Architecture Components

A.3.1 Training Pipeline

A.3.2 Inference Pipeline

A.4 Configuration Schema

The default configuration for AS-MoE-M (6.7B parameters):

Table 3: AS-MoE system requirements matrix.

ID	Requirement	Priority
SR-01	Support models up to 100B total parameters (13B active per token)	P0
SR-02	Training throughput $\geq$ 4000 tokens/sec/GPU on H100	P0
SR-03	Inference latency $\leq$ 30ms/token on single H100	P0
SR-04	Budget adherence: actual FLOPs within 5% of target	P0
SR-05	Support Expert Choice $\rightarrow$ Token Choice routing transition	P1
SR-06	Gradient mixing with DDP all-gather	P1
SR-07	NVSHMEM-based expert prefetching	P1
SR-08	Routing replay buffer for transfer learning	P2
SR-09	Dynamic topology (adaptive F, K)	P2
SR-10	Triton kernel for fused Grouped GEMM + GARN	P2

Table 4: Training pipeline components.

Component	Language	Description
ASMoELayer	Python/PyTorch	Core MoE layer with Core+LoRA, multi-channel routing, gradient mixing
BudgetController	Python/PyTorch	Lagrangian dual variable μ with EMA
GARNModule	Python/PyTorch	Gradient-adaptive router noise
RoutingReplayBuffer	Python/PyTorch	Mean/cov storage for transfer
FusedASMoEKernal	Triton	Fused Grouped GEMM + activation + GARN
CommOverlay	C++/CUDA	NVSHMEM-based expert prefetching

Table 5: Inference pipeline components.

Component	Language	Description
TokenChoiceRouter	C++/CUDA	Low-latency top-m routing with KV-cache
ExpertCache	C++	LRU cache for LoRA adapters
BatchScheduler	C++	Dynamic batching across generation levels

```python
# AS-MoE-M default configuration
config = {
    "model": {
        "d_model": 4096,
        "n_layers": 32,
        "n_heads": 32,
        "vocab_size": 32000,
    },
    "moe": {
        "G": 3,        # generations
        "F": 4,        # flavors
        "K": 2,        # channels
        "d_ffs": [1024, 2048, 4096],
        "budgets": [1.0, 2.0, 4.0],
        "lora_r": 8,
        "lora_alpha": 16.0,
        "top_m": 1,
        "d_proj": 2048,
    },
    "training": {
        "gamma_mix": 0.1,
        "target_budget": 2.0,
        "beta_ema": 0.99,
        "garn_sigma_base": 0.1,
        "garn_eta": 2.0,
        "garn_alpha": 0.5,
        "lambda_path": 0.01,
        "lambda_ewc": 1000.0,
        "lambda_replay": 0.1,
    },
    "system": {
        "dtype": "bfloat16",
        "kernel_fusion": True,
        "nvshmem_prefetch": True,
        "persistent_kernel": True,
    }
}
```

A.5 Milestones

Table 6: AS-MoE development milestones.

Phase	Timeline	Deliverable
M1	Week 1–4	PyTorch reference implementation (SR-01, SR-04)
M2	Week 5–8	Distributed training with DDP (SR-02, SR-06)
M3	Week 9–12	Triton kernel + inference optimization (SR-03, SR-10)
M4	Week 13–16	NVSHMEM integration + EC→TC transition (SR-05, SR-07)
M5	Week 17–20	Transfer learning pipeline (SR-08) + evaluation

B Notation Reference

Table 7: Summary of notation used throughout this paper.

Symbol	Domain	Description
G	$\mathbb{N}$	Number of generations
F	$\mathbb{N}$	Number of flavors
K	$\mathbb{N}$	Number of channels (views)
g, f, k	—	Generation, flavor, channel indices
d_{model}	$\mathbb{N}$	Model hidden dimension
$d_{\text{ff}}^{(g)}$	$\mathbb{N}$	FFN intermediate dimension for generation g
$\mathbf{W}^{\text{core}}$	$\mathbb{R}^{d\times d'}$	Core weight matrix (shared, frozen)
$\mathbf{A}, \mathbf{B}$	$\mathbb{R}^{d\times r}$, $\mathbb{R}^{r\times d'}$	LoRA adaptation matrices
r	$\mathbb{N}$	LoRA rank
α	$\mathbb{R}^+$	LoRA scaling factor
$\pi_g, \pi_f^{(k)}$	Δ^{G-1}, Δ^{F-1}	Routing distributions
$\mathbf{M}$	$\mathbb{R}^{N\times N}$	Gradient mixing matrix
$\mathbf{R}$	$\mathbb{R}^{N\times N}$	Activation correlation matrix
γ	$\mathbb{R}^+$	Gradient mixing strength
μ	$\mathbb{R}^+$	Lagrangian dual variable (budget)
β	$(0, 1)$	EMA decay rate
σ_{GARN}	$\mathbb{R}^+$	GARN noise scale
$H(\pi)$	$\mathbb{R}^+$	Entropy of routing distribution
$\mathcal{F}$	Functor	Compositional bridge $\mathbf{C}_{\text{quark}} \to \mathbf{C}_{\text{lang}}$

References

[1] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, "Adaptive mixtures of local experts," *Neural Computation*, vol. 3, no. 1, pp. 79–87, 1991.

[2] M. I. Jordan and R. A. Jacobs, "Hierarchical mixtures of experts and the EM algorithm," *Neural Computation*, vol. 6, no. 2, pp. 181–214, 1994.

[3] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proc. ICLR*, 2017.

[4] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *J. Mach. Learn. Res.*, vol. 23, no. 120, pp. 1–39, 2022.

[5] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, "GShard: Scaling giant models with conditional computation and automatic sharding," in *Proc. ICLR*, 2021.

[6] Y. Zhou, T. Lei, H. Liu, N. Du, Y. Huang, V. Zhao, A. Dai, Z. Chen, Q. Le, and J. Wei, "Mixture-of-experts with expert choice routing," in *Proc. NeurIPS*, 2022.

[7] D. Dai et al., "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.

[8] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.

[9] T. Gale, D. Elsen, and S. Hooker, "The state of sparse training in deep learning," in *Proc. ICML*, 2023.

[10] L. Arnold, *Random Dynamical Systems*. Springer, 2013.

[11] T. Kato, *Perturbation Theory for Linear Operators*. Springer, 2nd ed., 1980.

[12] B. Zoph, W. Fedus, L. Zettlemoyer, B. Li, J. Wei, and N. Shazeer, "ST-MoE: Designing stable and transferable sparse expert models," *arXiv preprint arXiv:2209.14589*, 2022.

[13] M. Lewis, S. Ghosh, and T. Liu, "BASE layers: Simplifying training of large, sparse models," in *Proc. ICML*, 2021.

[14] F. Wang et al., "IniLoRA: On the initialization of LoRA in large language models," *arXiv preprint*, 2024.

[15] Q. Shen et al., "PathMoE: Cross-layer routing for mixture-of-experts," *arXiv preprint*, 2024.

[16] H. Zhao et al., "FlashDMoE: Fast distributed MoE with persistent kernels," *arXiv preprint*, 2024.

[17] S. Kim et al., "SonicMoE: Enabling large-scale distributed MoE training with optimized kernel design," *arXiv preprint*, 2024.

[18] J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," *PNAS*, vol. 114, no. 13, pp. 3521–3526, 2017.

[19] P. Awasthi et al., "From sparse to structured: Rethinking MoE organization," *arXiv preprint*, 2024.

[20] S. Mac Lane, *Categories for the Working Mathematician*. Springer, 2nd ed., 1998.